
Table of Contents

Bijan Jourabchi	1
ASEN 6044 HW1 Q4	1
Load Datafile	1
Get Noise Covar matrix	1
Whitening transformation	1
NLS	1
Measurment function	3
Dynamics function	3
Jacobian fxn	3
Plotting Functions	4

Bijan Jourabchi

ASEN 6044 HW1 Q4

Load Datafile

```
clear; clc; close all
load("homework1_missileprob_data.mat")
```

Get Noise Covar matrix

```
N = size(ystacked,1)/2;
np = 2;
L = 80000;
dt = 5; % sec
t = dt:dt:N*dt;
```

```
R = [70 0; 0 0.005];
Rb = kron(eye(N), R);
```

Whitening transformation

```
S = chol(Rb);
ya = S*ystacked;
```

NLS

```
x0g = [10;40;2300;30];
%x0g = [130;4;2;6050];

% NLS
itr = 0;
maxitr = 100;
```

```

maxalph = 10;
converged = false;

while itr < maxitr && ~converged
    % Setup NLS
    yc = zeros(np,N);
    H = zeros(np*N,4);

    for k = 1:N

        yc(:,k) = h(x0g,t(k),R, L);

        % [xi,a] = f(x0g,t(k));
        H(2*k-1 : 2*k, :) = jcb(x0g,L,t(k));
    end
    yc = S*yc(:);
    H = S*H;

    % Cost fxn
    Jcurr = (ya - yc)'/Rb*(ya - yc);
    dx = inv(H'*inv(Rb)*H)*H'*inv(Rb)*(ya - yc);
    alph = 1;
    alphasplts = 0;
    while alphasplts < maxalph

        xn0 = x0g + alph*dx;

        yc = zeros(np,N);
        for k = 1:N
            yc(:,k) = h(xn0,t(k),R, L);
        end
        yc = S*yc(:);

        Jn = (ya - yc)'/Rb*(ya - yc);

        if abs(Jcurr - Jn) < 50
            converged = true;
            break;
        end

        if Jn > Jcurr
            alph = alph/2;
        else
            x0g = xn0;
            break;
        end
        alphasplts = alphasplts + 1;
    end
    itr = itr + 1;
end

plot_NLS(x0true,x0g,ystacked,t,Rb)

```

Measurment function

```
function [y] = h(x0,t,R, L)

    vk = mvnrnd([0,0],R)';

    [xik,ak] = f(x0,t);

    rho = sqrt((L - xik)^2 + ak^2) + vk(1);
    theta = atan2(ak, (L-xik)) + vk(2);

    y = [rho;theta];
end
```

Dynamics function

```
function [xik,ak] = f(x0,t)

    xi0 = x0(1);
    xid0 = x0(2);
    a0 = x0(3);
    ad0 = x0(4);
    g = 9.81;

    xik = xi0 + xid0*t;
    ak = a0 + ad0*t - 0.5*g*t^2;
end
```

Jacobian fxn

```
function [H] = jcb(x,L,t)

    xi = x(1);
    xid = x(2);
    a = x(3);
    ad = x(4);
    g = 9.81;

    H11 = 2*(xid*t+xi-L)/sqrt((2*a-g*t^2+2*ad*t)^2 + 4*(xid*t+xi-L)^2);
    H12 = H11 * t;
    H13 = (2*a - g*t^2 + 2*ad*t) / sqrt((2*a-g*t^2+2*ad*t)^2 + 4*(xid*t+xi-L)^2);
    H14 = H13*t;

    H21 = 2*(2*a - g*t^2 + 2*ad*t)/((2*a - g*t^2 + 2*ad*t)^2 + 4*(xid*t+xi-L)^2);
    H22 = H21*t;
    H23 = 4*(-xid*t-xi+L)/((2*a - g*t^2 + 2*ad*t)^2 + 4*(xid*t+xi-L)^2);
    H24 = -t*H23;
```

```

H = [H11 H12 H13 H14;H21 H22 H23 H24];
end

```

Plotting Functions

```

function [] = plot_NLS(x0t,x0NLS,Y,tvec,R)

N = length(tvec);

% Get Truth/Estimated Traj

trajT = zeros(2,N);
trajE = zeros(2,N);

for k = 1:N
    [xi,a] = f(x0t,tvec(k));
    trajT(:,k) = [xi;a];
    [xi,a] = f(x0NLS,tvec(k));
    trajE(:,k) = [xi;a];
end

% Get error and cov

for k = 1:N

    H(2*k-1 : 2*k, :) = jcb(x0NLS,80000,tvec(k));
end

P = inv(H'*inv(R)*H);

figure(); hold on; grid on
scatter(trajE(1,:),trajE(2,:), 's', 'MarkerFaceColor', [0.5 0 0.8],
'MarkerEdgeColor', [0.5 0 0.8])
scatter(trajT(1,:),trajT(2,:),100,'x','Color', '#40E0D0')
legend('NLS Estimate','Ground Truth')
xlabel("Easting (m)")
ylabel("Altitude (m)")
title("NLS Missile Tracking")

figure(); hold on; grid on
tiledlayout(2,1)
nexttile;
hold on
title('Error Time History in \zeta')
err1 = trajE(1,:)-trajT(1,:);
plot(tvec,err1,'k',LineWidth=1)
errorbar(tvec(1),err1(1),2*sqrt(P(1,1)), -2*sqrt(P(1,1)), 'horizontal', '-
o')
xlabel('Time')
ylabel('Error')

```

```

legend('Error', '2\sigma bounds')

nexttile;
hold on
title('Error Time History in a')
err2 = trajE(2,:)-trajT(2,:);
plot(tvec,err2,'k',LineWidth=1)
errorbar(tvec(1),err2(1),sqrt(P(2,2)),sqrt(P(2,2)),'-o')
xlabel('Time')
ylabel('Error')
legend('Error', '2\sigma bounds')

```

end

